

CAP 4611 Assignment 3

Name: [Jayden Rodriguez Lugo, David Navarrete]
Student number: [5505335, 5479357]

Important: Submission Format [5 points]

Please make sure to follow the submission instructions posted on the course website and on the webcourses. Here is a direct link to the instructions. [We will deduct marks if the submission format is incorrect.](#)

1 Matrix Notation and Minimizing Quadratics [12 points]

1.1 Converting to Matrix/Vector/Norm Notation [6 points]

Source: OpenAI Codex assistance with derivation, implementation, and explanation (October 6, 2026); assignment data and starter code supplied by the course.

Using our standard supervised learning notation (X, y, w) [express the following functions in terms of vectors, matrices, and norms](#) (there should be no summations or maximums).

1. $(\sum_{i=1}^n |w^T x_i - y_i|)^2$.

Answer: $\|Xw - y\|_1^2$

2. $\sum_{i=1}^n v_i (w^T x_i - y_i)^2 + \frac{\lambda}{2} \sum_{j=1}^d w_j^2$. This is regularized least squares with a *weight* v_i for each training example: Hint: You can use V to denote a diagonal matrix that has the values v_i along the diagonal. What does $a^T V b$ look like in summation form (for some arbitrary vectors a, b)?

Answer: $(Xw - y)^T V (Xw - y) + \frac{\lambda}{2} w^T w$. Here $r^T V r = \sum_i v_i r_i^2$.

3. $\max_{i \in \{1, 2, \dots, n\}} |w^T x_i - y_i| + \frac{1}{2} \sum_{j=1}^d \lambda_j |w_j|$. This is L1-regularized brittle regression with a different regularization strength for each dimension: Hint: You can use Λ to denote a diagonal matrix that has the λ_j values along the diagonal.

Answer: $\|Xw - y\|_\infty + 1/2 \|\Lambda w\|_1$

Note: you can assume that all the v_i and λ_i values are non-negative.

1.2 Minimizing Quadratic Functions as Linear Systems [6 points]

Source: OpenAI Codex assistance with derivation, implementation, and explanation (October 6, 2026); assignment data and starter code supplied by the course.

[Write finding a minimizer \$w\$ of the functions below as a system of linear equations](#) (using vector/matrix notation and simplifying as much as possible). Note that all the functions below are convex, so finding a w with $\nabla f(w) = 0$ is sufficient to minimize the functions – but show your work in getting to this point.

1. $f(w) = \frac{1}{2} \|w - v\|^2$ (projection of v onto real space).

Answer: Expanding the square and differentiating gives

$$f(w) = \frac{1}{2}(w - v)^T(w - v), \quad \nabla f(w) = w - v.$$

Setting the gradient equal to zero yields $w - v = 0$, or

$$\boxed{Iw = v}.$$

2. $f(w) = \frac{1}{2}\|Xw - y\|^2 + \frac{1}{2}w^T\Lambda w$ (least squares with weighted regularization).

Answer: Expanding the least-squares term and differentiating (with diagonal, symmetric Λ) gives

$$\begin{aligned} f(w) &= \frac{1}{2}(Xw - y)^T(Xw - y) + \frac{1}{2}w^T\Lambda w, \\ \nabla f(w) &= X^T(Xw - y) + \Lambda w \\ &= X^T Xw - X^T y + \Lambda w. \end{aligned}$$

Setting $\nabla f(w) = 0$ and collecting terms gives

$$\boxed{(X^T X + \Lambda)w = X^T y}.$$

3. $f(w) = \frac{1}{2}\sum_{i=1}^n v_i(w^T x_i - y_i)^2 + \frac{\lambda}{2}\|w - w^{(0)}\|^2$ (weighted least squares shrunk towards non-zero $w^{(0)}$).

Answer: Let $V = \text{diag}(v_1, \dots, v_n)$. Then

$$f(w) = \frac{1}{2}(Xw - y)^T V (Xw - y) + \frac{\lambda}{2}(w - w^{(0)})^T(w - w^{(0)}).$$

Differentiating and using the symmetry of V gives

$$\begin{aligned} \nabla f(w) &= X^T V (Xw - y) + \lambda(w - w^{(0)}) \\ &= X^T V Xw - X^T V y + \lambda w - \lambda w^{(0)}. \end{aligned}$$

Setting this to zero and collecting terms gives

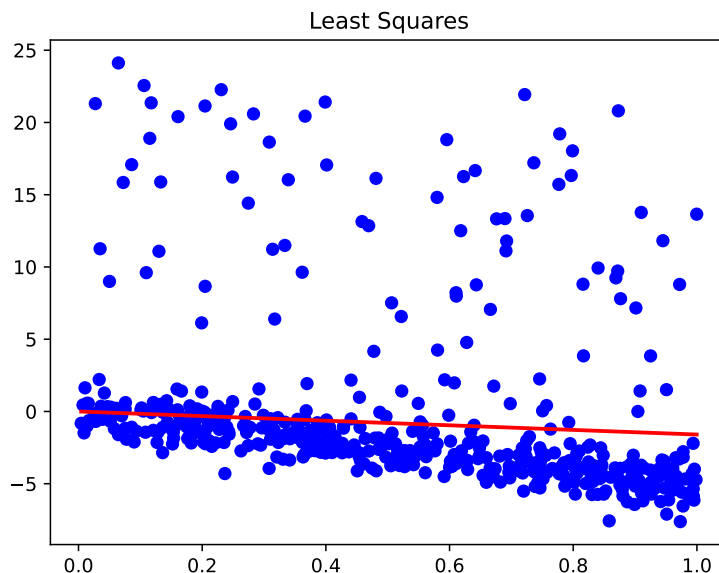
$$\boxed{(X^T V X + \lambda I)w = X^T V y + \lambda w^{(0)}}.$$

Above we assume that v and $w^{(0)}$ are $d \times 1$ vectors, and Λ is a $d \times d$ diagonal matrix (with positive entries along the diagonal). You can use V as a diagonal matrix containing the v_i values along the diagonal.

Hint: Once you convert to vector/matrix notation, you can use the results from class to quickly compute these quantities term-wise. As a spot check, make sure that the dimensions match for all quantities/operations: to do this, you may need to introduce an identity matrix. For example, $X^T Xw + \lambda w$ can be re-written as $(X^T X + \lambda I)w$.

2 Robust Regression and Gradient Descent [41 points]

If you run `python main.py 2`, it will load a one-dimensional regression dataset that has a non-trivial number of ‘outlier’ data points. These points do not fit the general trend of the rest of the data, and pull the least squares model away from the main downward trend that most data points exhibit:



Note: we are fitting the regression without an intercept here, just for simplicity of the homework question. In reality one would rarely do this. But here it’s OK because the “true” line passes through the origin (by design). In Q3.1 we’ll address this explicitly.

A coding note: when we’re doing math, we always treat y and w as column vectors, i.e. if we’re thinking of them as matrices, then shape $n \times 1$ or $d \times 1$, respectively. This is also what you’d usually do when coding things in, say, Matlab. It is *not* what’s usually done in Python machine learning code, though: we usually have `y.shape == (n,)`, i.e. a one-dimensional array. Mathematically, these are the same thing, but if you mix between the two, you can really easily get confusing answers: if you add something of shape `(n, 1)` to something of shape `(n,)`, then the NumPy broadcasting rules give you something of shape `(n, n)`. This is a very unfortunate consequence of the way the broadcasting rules work. If you stick to either one, you generally don’t have to worry about it; **we’re assuming shape `(n,)` here**. Note that you can ensure you have something of shape `(n,)` with the `utils.ensure_1d` helper, which basically just uses `two_d_array.squeeze(1)` (which checks that the axis at index 1, the second one, is length 1 and then removes it). You can go from `(n,)` to `(n, 1)` with, for instance, `one_d_array[:, np.newaxis]` (which says “give me the whole first axis, then add another axis of length 1 in the second position”).

2.1 Weighted Least Squares in One Dimension [8 points]

Source: OpenAI Codex assistance with derivation, implementation, and explanation (October 6, 2026); assignment data and starter code supplied by the course.

One of the most common variations on least squares is *weighted* least squares. In this formulation, we have

a weight v_i for every training example. To fit the model, we minimize the weighted squared error,

$$f(w) = \frac{1}{2} \sum_{i=1}^n v_i (w^T x_i - y_i)^2.$$

In this formulation, the model focuses on making the error small for examples i where v_i is high. Similarly, if v_i is low then the model allows a larger error. Note: these weights v_i (one per training example) are completely different from the model parameters w_j (one per feature), which, confusingly, we sometimes also call “weights.” The v_i are sometimes called *sample weights* or *instance weights* to help distinguish them.

Complete the model class, `WeightedLeastSquares` (inside `linear_models.py`), to implement this model. (Note that Q1.2.3 asks you to show how a similar formulation can be solved as a linear system.) Apply this model to the data containing outliers, setting $v = 1$ for the first 400 data points and $v = 0.1$ for the last 100 data points (which are the outliers). [Hand in your code and the updated plot.](#)

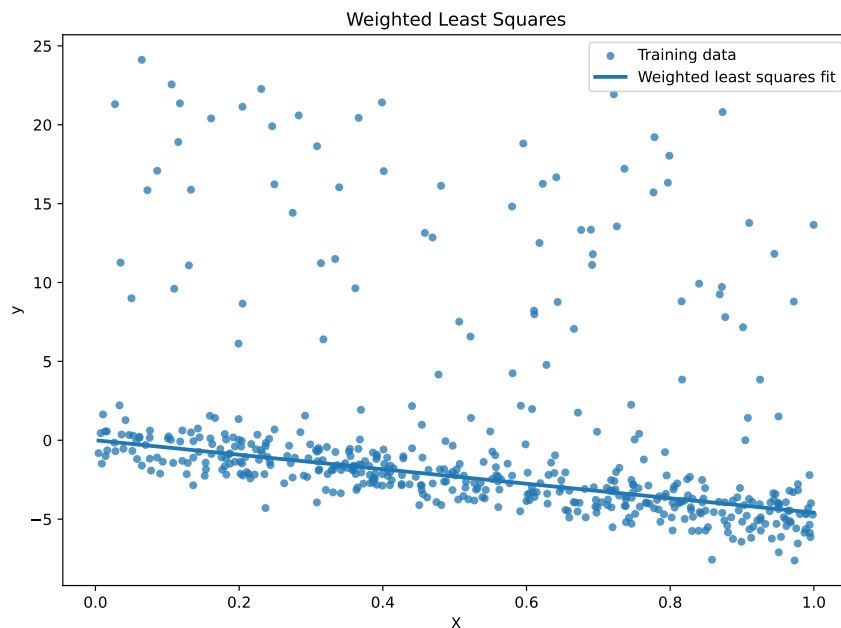
Answer:

```
1 class WeightedLeastSquares(LeastSquares):
2     def fit(self, X, y, v):
3         #take the square root of v
4         sqrt_v = np.sqrt(v)
5
6         #get weighted versions of X, and y
7         weighted_X = X * sqrt_v[:, None]
8         weighted_y = np.asarray(y).reshape(-1) * sqrt_v
9
10        #solve weighted least squares
11        self.w = np.linalg.lstsq(weighted_X, weighted_y, rcond=None)[0]

```



```
1 v = np.ones(len(y))
2 v[400:] = 0.1
3 model = WeightedLeastSquares()
4 model.fit(X, y, v)
5 test_and_plot(model, X, y, title="Weighted Least Squares",
6               filename="Weighted_least_squares_outliers_Q2.1.pdf")
```



The fitted slope is $w = -4.592700$. The unweighted training MSE is 40.8732; it is slightly larger than ordinary least squares because the weighted model prioritizes the first 400 examples rather than the unweighted squared error over all 500 examples.

2.2 Smooth Approximation to the L1-Norm [8 points]

Source: OpenAI Codex assistance with derivation, implementation, and explanation (October 6, 2026); assignment data and starter code supplied by the course.

Unfortunately, we typically do not know the identities of the outliers. In situations where we suspect that there are outliers, but we do not know which examples are outliers, it makes sense to use a loss function that is more robust to outliers. In class, we discussed using the sum of absolute values objective,

$$f(w) = \sum_{i=1}^n |w^T x_i - y_i|.$$

This is less sensitive to outliers than least squares, but it is non-differentiable and harder to optimize. Nevertheless, there are various smooth approximations to the absolute value function that are easy to optimize. One possible approximation is to use the log-sum-exp approximation of the max function¹:

$$|r| = \max\{r, -r\} \approx \log(\exp(r) + \exp(-r)).$$

Using this approximation, we obtain an objective of the form

$$f(w) = \sum_{i=1}^n \log(\exp(w^T x_i - y_i) + \exp(y_i - w^T x_i)).$$

which is smooth but less sensitive to outliers than the squared error. Derive the gradient ∇f of this function with respect to w . You should show your work but you do not have to express the final result in matrix notation.

¹Other possibilities are the Huber loss, or $|r| \approx \sqrt{r^2 + \epsilon}$ for some small ϵ .

Answer: Let $r_i = w^T x_i - y_i$, so $\nabla_w r_i = x_i$. By the chain rule,

$$\nabla_w \log(e^{r_i} + e^{-r_i}) = \frac{e^{r_i} - e^{-r_i}}{e^{r_i} + e^{-r_i}} x_i = \tanh(r_i) x_i.$$

Summing over examples gives

$$\nabla f(w) = \sum_{i=1}^n \tanh(w^T x_i - y_i) x_i = X^T \tanh(Xw - y).$$

The hyperbolic tangent acts elementwise in the matrix expression.

2.3 Gradient Descent: Understanding the Code [5 points]

Source: OpenAI Codex assistance with derivation, implementation, and explanation (October 6, 2026); assignment data and starter code supplied by the course.

Recall gradient descent, a derivative-based optimization algorithm that uses gradients to navigate the parameter space until a locally optimal parameter is found. In `optimizers.py`, you will see our implementation of gradient descent, taking the form of a class named `GradientDescent`. This class has a similar design pattern as PyTorch, a popular differentiable programming and optimization library. One step of gradient descent is defined as

$$w^{t+1} = w^t - \alpha^t \nabla_w f(w^t).$$

Look at the methods named `get_learning_rate_and_step()` and `break_yes()`, and answer each of these questions, one sentence per answer:

1. Which variable is equivalent to α^t , the step size at iteration t ?

Answer: The step size α^t is represented by the variable `$\alpha$` .

2. Which variable is equivalent to $\nabla_w f(w^t)$ the current value of the gradient vector?

Answer: The current gradient $\nabla_w f(w^t)$ is represented by `$\mathbf{g\_old}$` .

3. Which variable is equivalent to w^t , the current value of the parameters?

Answer: The current parameter vector w^t is represented by `$\mathbf{w\_old}$` .

4. What is the method `break_yes()` doing?

Answer: The method `break_yes()` checks whether the stopping conditions have been reached, such as the optimality tolerance or the permitted evaluation limit, and returns true when optimization should stop.

2.4 Robust Regression [20 points]

The class `LinearModel` is like `LeastSquares`, except that it fits the least squares model using a gradient descent method. If you run `python main.py 2.4` you'll see it produces the same fit as we obtained using the normal equations.

The typical input to a gradient method is a function that, given w , returns $f(w)$ and $\nabla f(w)$. See `fun_obj.py` for some examples. Note that the `fit` function of `LinearModel` also has a numerical check that the gradient code is approximately correct, since implementing gradients is often error-prone.²

²Sometimes the numerical gradient checker itself can be wrong. See <https://math.umd.edu/~dlevy/classes/amsc466/lecture-notes/differentiation-chap.pdf> for a lot more on numerical differentiation.

An advantage of gradient-based strategies is that they are able to solve problems that do not have closed-form solutions, such as the formulation from the previous section. The class `LinearModel` has most of the implementation of a gradient-based strategy for fitting the robust regression model under the log-sum-exp approximation.

2.4.1 Implementing the Objective Function [15 points]

Source: OpenAI Codex assistance with derivation, implementation, and explanation (October 6, 2026); assignment data and starter code supplied by the course.

Optimizing robust regression parameters is the matter of implementing a function object and using an optimizer to minimize the function object. The only part missing is the function and gradient calculation inside `fun_obj.py`. Inside `fun_obj.py`, complete `RobustRegressionLoss` to implement the objective function and gradient based on the smooth approximation to the absolute value function (from the previous section). Hand in your code, as well as the plot obtained using this robust regression approach.

Answer:

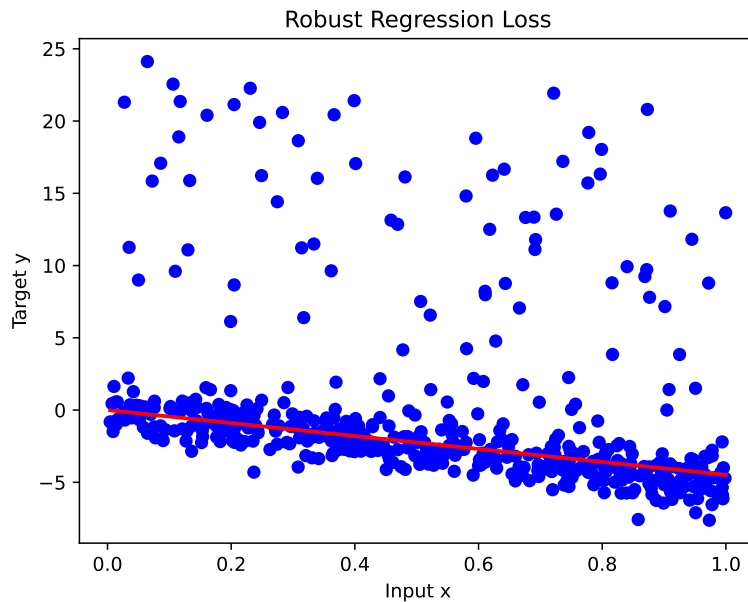
```
1 class RobustRegressionLoss(FunObj):
2     def evaluate(self, w, X, y):
3         w = ensure_1d(w)
4         y = ensure_1d(y)
5
6         #predictions, residuals
7         resid = (X @ w) - y
8
9         #robust loss
10        f_sum = np.sum(np.logaddexp(resid, -resid))
11
12        #gradient
13        gradient = X.T @ np.tanh(resid)
14
15        return f_sum, gradient

```



```
1 fun_obj = RobustRegressionLoss()
2 optimizer = GradientDescentLineSearch()
3 model = LinearModel(fun_obj, optimizer)
4 model.fit(X, y)
5 test_and_plot(model, X, y, title="Robust Regression",
6               filename="Robust_Regression_Loss_Q2.4.pdf")

```



The unweighted training MSE is 40.6 (rounded to one decimal place). This model minimizes the smooth robust objective rather than squared error. A finite-difference check agrees with the analytic gradient to absolute tolerance 10^{-3} .

2.4.2 The Learning Curves [5 points]

Source: OpenAI Codex assistance with derivation, implementation, and explanation (October 6, 2026); assignment data and starter code supplied by the course.

Using the same dataset as the previous sections, produce the plot of “gradient descent learning curves” to compare the performances of `GradientDescent` and `GradientDescentLineSearch` for robust regression, where **one hundred (100) iterations** of gradient descent are on the x-axis and the **objective function value** corresponding to each iteration is visualized on the y-axis (see gradient descent lecture). Use the default `learning_rate` for `GradientDescent`. Submit this plot. According to this plot, which optimizer is more “iteration-efficient”?

Answer:

```

1 def q2_4_2():
2     # loads the data in the form of dictionary
3     data = load_dataset("outliersData.pkl")
4     X = data["X"]
5     y = data["y"].squeeze(1)
6
7     # -----
8     # Gradient Descent
9     # -----
10    fun_obj = RobustRegressionLoss()
11
12    optimizer = GradientDescent(
13        optimal_tolerance=0,
14        max_evals=100,
15        verbose=False

```

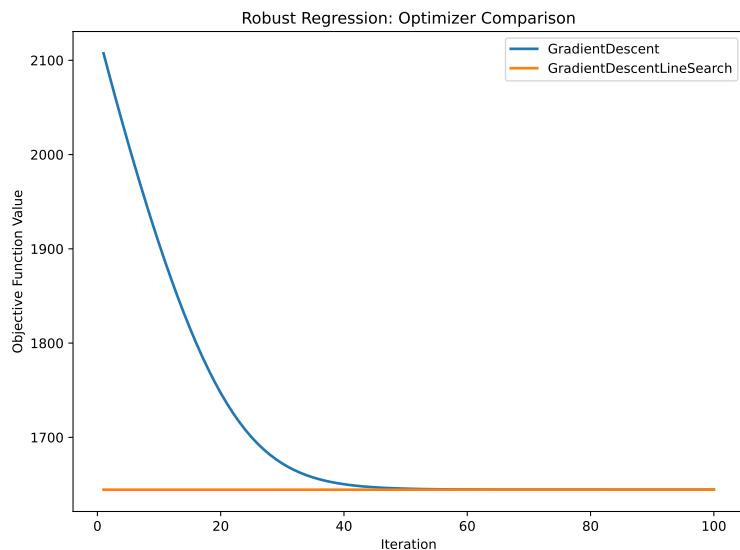


```

16     )
17
18     model_gd = LinearModel(fun_obj, optimizer)
19     model_gd.fit(X, y)
20
21     # Store objective values
22     gd_values = model_gd.fs
23
24     # -----
25     # Gradient Descent Line Search
26     # -----
27     optimizer_ls = GradientDescentLineSearch(
28         optimal_tolerance=0,
29         max_evals=100,
30         verbose=False
31     )
32
33     model_ls = LinearModel(fun_obj, optimizer_ls)
34     model_ls.fit(X, y)
35
36     # Store objective values
37     ls_values = model_ls.fs
38
39     # -----
40     # Plot learning curves
41     # -----
42     iterations_gd = range(len(gd_values))
43     iterations_ls = range(len(ls_values))
44
45     plt.figure()
46
47     plt.plot(
48         iterations_gd,
49         gd_values,
50         label="GradientDescent"
51     )
52
53     plt.plot(
54         iterations_ls,
55         ls_values,
56         label="GradientDescentLineSearch"
57     )
58
59     plt.xlabel("Iteration")
60     plt.ylabel("Objective Function Value")
61     plt.title("Robust Regression: Optimizer Comparison")
62
63     plt.legend()
64
65     plt.savefig(
66         Path("..", "figs", "Robust_Regression_Optimizer_Comparison_Q2.4.2.pdf")
67     )
68
69     plt.close()

```

Answer:



Iteration	Fixed-step GD objective	Line-search objective
0	2132.555438	2132.555438
1	2107.394652	1644.520444
5	2011.950541	1644.492663
10	1905.470767	1644.492663
100	1644.492835	1644.492663

The line-search optimizer reaches the low objective region in fewer recorded updates, so it is more iteration-efficient in the reported comparison. The plotting code uses the optimizer's recorded objective values; check whether `max_evals=100` corresponds to 100 parameter updates in the supplied implementation before labeling the final row as iteration 100. Iteration efficiency does not necessarily imply fewer objective-function evaluations, since line search may test multiple step sizes.

3 Linear Regression and Nonlinear Bases

In class we discussed fitting a linear regression model by minimizing the squared error. In this question, you will start with a data set where least squares performs poorly. You will then explore how adding a bias variable and using nonlinear (polynomial) bases can drastically improve the performance. You will also explore how the complexity of a basis affects both the training error and the validation error.

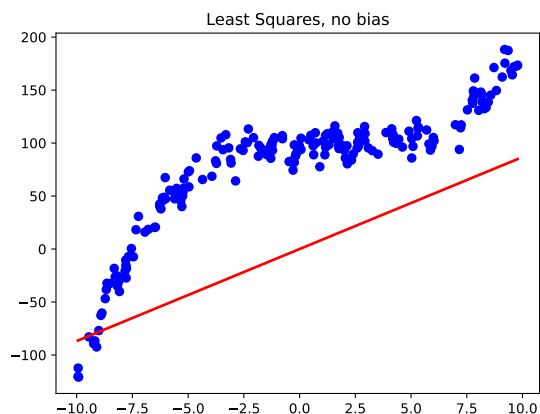
3.1 Adding a Bias Variable [8 points]

Source: OpenAI Codex assistance with derivation, implementation, and explanation (October 6, 2026); assignment data and starter code supplied by the course.

If you run `python main.py 3`, it will:

1. Load a one-dimensional regression dataset.
2. Fit a least-squares linear regression model.
3. Report the training error.
4. Report the validation error.
5. Draw a figure showing the training data and what the linear model looks like.

Unfortunately, this is an awful model of the data. The average squared training error on the data set is over 7000 (as is the validation error), and the figure produced by the demo confirms that the predictions are usually nowhere near the training data:



The y -intercept of this data is clearly not zero (it looks like it's closer to 200), so we should expect to improve performance by adding a *bias* (a.k.a. intercept) variable, so that our model is

$$y_i = w^T x_i + w_0.$$

instead of

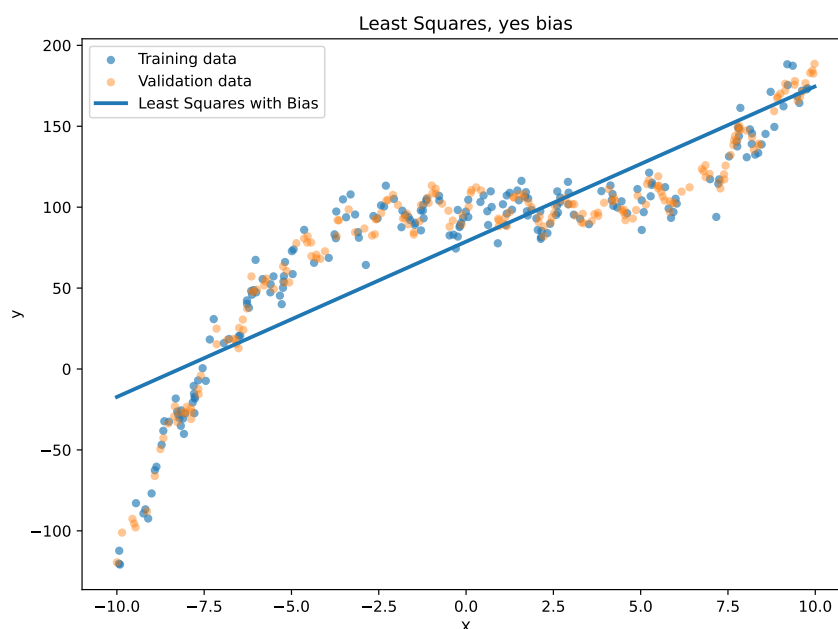
$$y_i = w^T x_i.$$

In file `linear_models.py`, complete the class `LeastSquaresBias`, that has the same input/model/predict format as the `LeastSquares` class, but that adds a *bias* variable (also called an intercept) w_0 (also called β in lecture). Hand in your new class, the updated plot, and the updated training/validation error.

Hint: recall that adding a bias w_0 is equivalent to adding a column of ones to the matrix X . Don't forget that you need to do the same transformation in the `predict` function.

Answer:

```
1 class LeastSquaresBias:
2     "Least Squares with a bias added"
3
4     def fit(self, X, y):
5         Z = np.column_stack((np.ones(X.shape[0]), X))
6         self.w = np.linalg.lstsq(Z, np.asarray(y).reshape(-1), rcond=None)[0]
7
8     def predict(self, X_pred):
9         Z = np.column_stack((np.ones(X_pred.shape[0]), X_pred))
10        return Z @ self.w
```



The fitted model is $\hat{y} = 78.712232 + 9.597789x$. Training MSE is 938.369 and validation MSE is 844.378, compared with 7107.2 and 7085.6 without a bias. The intercept improves the fit substantially, but the remaining error motivates a nonlinear basis.

3.2 Polynomial Basis [10 points]

Source: OpenAI Codex assistance with derivation, implementation, and explanation (October 6, 2026); assignment data and starter code supplied by the course.

Adding a bias variable improves the prediction substantially, but the model is still problematic because the target seems to be a *non-linear* function of the input. Complete `LeastSquarePoly` class, that takes a data vector x (i.e., assuming we only have one feature) and the polynomial order p . The function should perform a least squares fit based on a matrix Z where each of its rows contains the values $(x_i)^j$ for $j = 0$ up to p . E.g., `LeastSquaresPoly.fit(x,y)` with $p = 3$ should form the matrix

$$Z = \begin{bmatrix} 1 & x_1 & (x_1)^2 & (x_1)^3 \\ 1 & x_2 & (x_2)^2 & (x_2)^3 \\ \vdots & & & \\ 1 & x_n & (x_n)^2 & (x_n)^3 \end{bmatrix},$$

and fit a least squares model based on it. Submit your code, and a plot showing training and validation error curves for the following values of p : 0, 1, 2, 3, 4, 5, 10, 20, 30, 50, 75, 100. Clearly label your axes, and use a logarithmic scale for y by `plt.yscale("log")` or similar, so that we can still see what's going on if there are a few extremely large errors. Explain the effect of p on the training error and on the validation error.

NOTE: large values of p may cause numerical instability. Your solution may look different from others' even with the same code depending on the OS and other factors. As long as your training and validation error curves behave as expected, you will not be penalized.

Note: you should write the code yourself; don't use a library like sklearn's `PolynomialFeatures`.

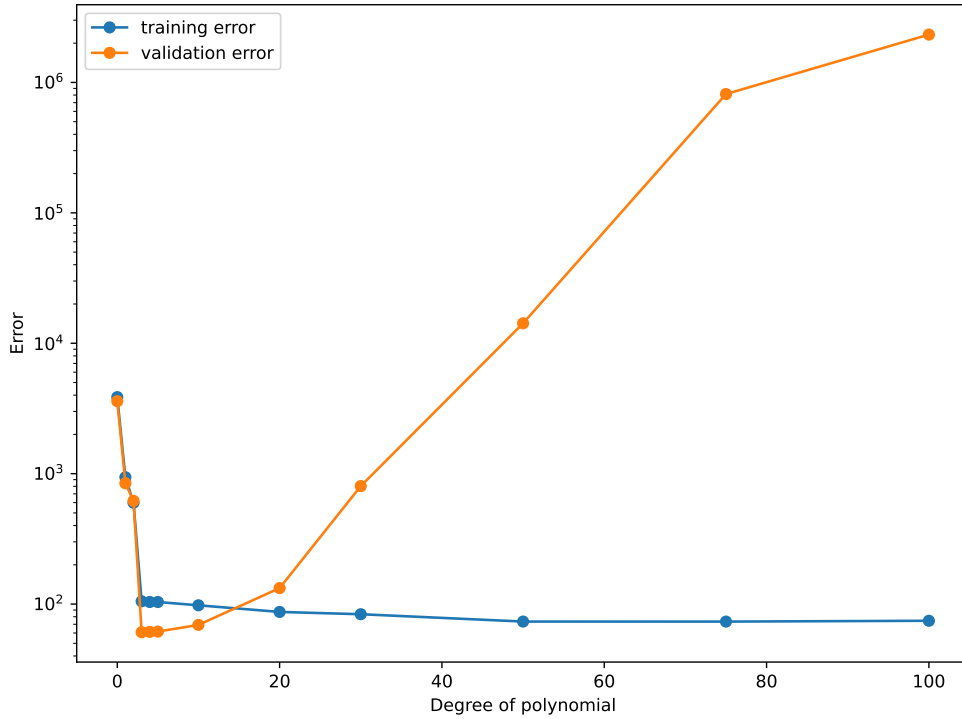
Note: in addition to the error curves, the code also produces a plot of the fits themselves. This is for your information; you don't have to submit it.

Answer:

```
1 class LeastSquaresPoly:
2     "Least Squares with polynomial basis"
3
4     def __init__(self, p):
5         self.p = p
6
7     def fit(self, X, y):
8         Z = self._poly_basis(X)
9         self.w = np.linalg.lstsq(Z, np.asarray(y).reshape(-1), rcond=None)[0]
10
11     def predict(self, X_pred):
12         return self._poly_basis(X_pred) @ self.w
13
14     def _poly_basis(self, X):
15         x = np.asarray(X).reshape(-1)
16         return x[:, None] ** np.arange(self.p + 1)[None, :]
```



```
1 p_vals = [0, 1, 2, 3, 4, 5, 10, 20, 30, 50, 75, 100]
2 err_trains, err_valids = [], []
3 for p in p_vals:
4     model = LeastSquaresPoly(p)
5     model.fit(X, y)
6     err_trains.append(np.mean((model.predict(X) - y)**2))
7     err_valids.append(np.mean((model.predict(X_valid) - y_valid)**2))
8 plt.plot(p_vals, err_trains, marker="o", label="Training MSE")
9 plt.plot(p_vals, err_valids, marker="o", label="Validation MSE")
10 plt.xlabel("Polynomial degree p")
11 plt.ylabel("Mean squared error")
12 plt.yscale("log")
13 plt.legend()
14 plt.savefig("../figs/polynomial_error_curves.pdf")
```



Degree p	Training MSE	Validation MSE
0	3862.06	3594.01
1	938.369	844.378
2	598.711	618.364
3	105.067	60.8836
4	103.744	61.2340
5	103.730	61.5202
10	97.8467	69.2740
20	4963.23	6248.39
30	5656.20	14723.9
50	6894.11	272057
75	7558.90	1.50931×10^6
100	7830.50	1.42898×10^7

Degree 3 has the smallest validation MSE among the degrees tested in this run. In exact arithmetic, the minimum training error cannot increase when the degree increases, since each polynomial space contains the preceding one. Low degrees underfit; moderately larger degrees capture the nonlinear pattern. High degrees can overfit and worsen validation error. Raw monomial columns also become extremely ill-conditioned at large degrees, so numerical rank truncation can cause the computed training error to increase as well. The solver uses `lstsq`, avoiding the additional conditioning damage of forming normal equations, but it does not eliminate instability in the raw basis.

4 Very-Short Answer Questions [24 points]

Source: OpenAI Codex assistance with derivation, implementation, and explanation (October 6, 2026); assignment data and starter code supplied by the course.

Answer the following questions (in a sentence or two).

1. Suppose that a training example is global outlier, meaning it is really far from all other data points. How is the cluster assignment of this example set by k -means? And how is it set by density-based clustering?

Answer: k -means assigns the point to its nearest centroid, even if that centroid is far away; it can also pull a centroid toward itself. A density-based method such as DBSCAN labels it as noise if it has insufficient neighbors to be a core point and is not reachable from a core point.

2. Why do we need random restarts for k -means but not for density-based clustering?

Answer: k -means depends on its initial centroids and may converge to different local minima, so random restarts improve the chance of a good solution. Standard DBSCAN uses density connectivity rather than random initial centroids, so random restarts are unnecessary (although border assignments may depend on traversal order).

3. Can hierarchical clustering find non-convex clusters?

Answer: Yes. Single-linkage hierarchical clustering can recover non-convex clusters by linking nearby points along their shapes, although it can suffer from chaining; other linkage rules need not recover these shapes.

4. For model-based outlier detection, list an example method and problem with identifying outliers using this method.

Answer: Fit a Gaussian distribution and flag points with large Mahalanobis distance or low fitted density. If the data are multimodal or non-Gaussian, the assumed model can incorrectly flag ordinary observations; outliers can also distort the fitted parameters.

5. For graphical-based outlier detection, list an example method and problem with identifying outliers using this method.

Answer: Use a boxplot and flag points beyond the interquartile-range fences. Skewed data can produce false positives, and inspecting each coordinate separately can miss multivariate outliers.

6. For supervised outlier detection, list an example method and problem with identifying outliers using this method.

Answer: Train a binary classifier on labeled normal and anomalous examples. Anomalies are often rare and labels expensive, and the classifier may miss new types of anomalies absent from its training set.

7. If we want to do linear regression with 1 feature, explain why it would or would not make sense to use gradient descent to compute the least squares solution.

Answer: With one feature, a closed-form least-squares solution is inexpensive, so gradient descent usually adds unnecessary iteration and step-size choices. It can still be useful for streaming data or very large datasets that favor incremental updates.

8. Why do we typically add a column of 1 values to X when we do linear regression? Should we do this if we're using decision trees?

Answer: A column of ones provides an intercept, allowing nonzero predictions when the original features are zero. Decision trees already predict constants in their leaves, and a constant input cannot provide a useful split, so this column is unnecessary.

9. Why do we need gradient descent for the robust regression problem, as opposed to just using the normal equations? Hint: it is NOT because of the non-differentiability. Recall that we used gradient descent even after smoothing away the non-differentiable part of the loss.

Answer: The robust gradient contains $\tanh(Xw - y)$, making the stationarity equation nonlinear in w . Normal equations solve the quadratic least-squares objective, whereas this smooth nonquadratic objective generally requires iterative optimization such as gradient descent.

10. What is the problem with having too small of a learning rate in gradient descent? What is the problem with having too large of a learning rate in gradient descent?

Answer: A very small learning rate gives tiny updates and slow convergence. A very large learning rate can overshoot, oscillate, or diverge.

11. What is the purpose of the log-sum-exp function and how is this related to gradient descent?

Answer: Log-sum-exp smoothly approximates a maximum; here $\log(e^r + e^{-r})$ approximates $\max(r, -r) = |r|$. Its differentiability supplies a usable gradient for gradient descent while retaining a robust, approximately absolute-value loss.

12. What type of non-linear transform might be suitable if we had a periodic function?

Answer: A Fourier basis of sine and cosine terms is suitable, for example $\sin(2\pi kx/T)$ and $\cos(2\pi kx/T)$ for a period T and several harmonics k .